

2025-2026 春 《数据结构与算法》 第三次实验题解

助教-陈孙一硕

A 杂货店店主的烦恼 II

一个订单要么不做，要么再其过期之前，成本最低的那天去做。注意某天做了一个订单之后，那天的成本会受影响。用堆模拟即可。

```
//#include<bits/stdc++.h>
#include<iostream>
#include<iomanip>
#include<vector>
#include<string>
#include<cmath>
#include<algorithm>
#include<set>
#include<map>
#include<unordered_map>
#include<unordered_set>
#include<queue>
#include<stack>
using namespace std;
#define int long long
#define endl '\n'
//const int mod=1e9+7;
const int mod=998244353;
const int N=2e6+101;
int a[N];
pair<bool,int> q[N];
int ans[N];
void solve(){
```

```

int n,m,Q; cin>>n>>m>>Q;
for (int i=0;i<=n+1;i++) q[i].first=0,q[i].second=0;
for (int i=1;i<=n;i++) cin>>a[i];
for (int i=1;i<=m;i++){
    int r,w; cin>>r>>w;
    q[r].first=1,q[r].second+=w;
}
priority_queue<int,vector<int>,greater<int>> pq;
for (int i=1;i<=n;i++){
    pq.push(a[i]);
    if (q[i].first){
        int t=pq.top();
        pq.pop();
        pq.push(t-q[i].second);
    }
}
int sum=0,idx=1;
while (!pq.empty()){
    int t=pq.top();
    pq.pop();
    sum+=t;
    ans[idx]=sum;
    idx++;
}
while (Q--){
    int k; cin>>k;
    cout<<-ans[k]<<endl;
}
}
signed main(){
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cout.tie(nullptr);
    int T=1; //cin>>T;
    while (T--) solve();
}

```

B 杂货店店主的烦恼 III

显然进货越多，赚钱越多，这是单调的。自然地想到二分答案后进行 check。在二分内部 check 的时候必须采用插入的做法，不能直接排序，否则复杂度多一个 $\log$ ，会被卡掉。

```
//#include<bits/stdc++.h>
#include<iostream>
#include<iomanip>
#include<vector>
#include<string>
#include<cmath>
#include<algorithm>
#include<set>
#include<map>
#include<unordered_map>
#include<unordered_set>
#include<queue>
#include<stack>
using namespace std;
#define int long long
#define endl '\n'
//const int mod=1e9+7;
const int mod=998244353;
const int N=1e7+101;
int a[N],b[N];
int n,m;
bool check(int mid){
    __int128 res=0;
    int j=1;
    bool flag=0;
    for (int i=1;i<=n+1;i++){
        if (flag) res+=a[j]*b[i],j++;
        else if (mid>a[j]) res+=mid*b[i],flag=1;
        else res+=a[j]*b[i],j++;
        if (res>=m) return 1;
    }
    return 0;
}
```

```

}
void solve(){
    cin>>n>>m;
    for (int i=0;i<=n+2;i++) a[i]=b[i]=0;
    for (int i=1;i<=n;i++) cin>>a[i];
    for (int i=1;i<=n+1;i++) cin>>b[i];
    sort(a+1,a+n+1,greater<int>());
    sort(b+1,b+n+2,greater<int>());
    int l=0,r=1e9;
    while (l<=r){
        int mid=(l+r)>>1;
        if (check(mid)) r=mid-1;
        else l=mid+1;
    }
    cout<<l<<endl;
}
signed main(){
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cout.tie(nullptr);
    int T=1; cin>>T;
    while (T--) solve();
}

```

C 序列操作

其实就是求最长严格上升子序列的长度。使用二分优化即可。这里使用了 `set`，其实使用 `vector` 也行。

```

#include<iostream>
#include<set>
using namespace std;
int num,n;
set<int> s;
int main(){
    cin>>n;

```

```

    for(int i=0;i<n;i++){
        cin>>num;
        if(s.lower_bound(num)!=s.end()) s.erase(s.lower_bound(num));
        s.insert(num);
    }
    cout<<s.size()<<endl;
    return 0;
}

```

D 后序和中序构造二叉树

模板题，基本所有人都通过了，没什么好说的。不过仅仅会抄板子是不够的，最好是能自己写出来，毕竟考试可是闭卷。

```

#include<iostream>
using namespace std;
#define int long long
int n,post[15],in[15];
struct node{
    int val;
    node* lchild=NULL,*rchild=NULL;
};
node* make(int lin,int rin,int lpost,int rpost){
    node* nd=new node;
    nd->val=post[rpost];
    int k=0;
    while (in[lin+k]!=post[rpost]) k++;
    if (k>0) nd->lchild=make(lin,lin+k-1,lpost,lpost+k-1);
    if (lin+k<rin) nd->rchild=make(lin+k+1,rin,lpost+k,rpost-1);
    return nd;
}
void preorder(node* nd){
    if (nd==NULL) return;
    cout<<nd->val<<" ";
    preorder(nd->lchild),preorder(nd->rchild);
}

```

```

signed main(){
    cin>>n;
    for (int i=1;i<=n;i++) cin>>post[i];
    for (int i=1;i<=n;i++) cin>>in[i];
    node* root=make(1,n,1,n);
    preorder(root);
    return 0;
}

```

E 奇偶平衡子段

非常经典的一个问题。把奇数当成 1，偶数当成-1，问题就转化成找一个和为 0 的最长子段。从前向后扫描，将前缀和及其对应的下标放入 set 中，每次插入之前查询更新答案即可。

```

#include<iostream>
#include<iomanip>
#include<vector>
#include<string>
#include<cmath>
#include<algorithm>
#include<set>
#include<map>
#include<unordered_map>
#include<unordered_set>
#include<queue>
#include<stack>
#include<fstream>
#define int long long
using namespace std;
const int N=2e6+101;
int a[N];
int pre[N];
void solve(){
    int n; cin>>n;
    for (int i=0;i<=n+1;i++) pre[i]=0;
}

```

```

for (int i=1;i<=n;i++) cin>>a[i];
for (int i=1;i<=n;i++){
    pre[i]=pre[i-1];
    if (a[i]&1) pre[i]++;
    else pre[i]--;
}
set<pair<int,int>> st;//fi:val se:idx
st.insert({0,0});
int ans=-1;
for (int i=1;i<=n;i++){
    auto it=st.lower_bound({pre[i],-1});
    if (it!=st.end()&&it->first==pre[i]) ans=max(ans,i-it->second);
    st.insert({pre[i],i});
}
cout<<ans<<endl;
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int T=1; cin>>T;
    while (T--) solve();
    return 0;
}

```

F 二叉查找树的构建及序列化

模板题，没啥好说的。

```

#include<iostream>
using namespace std;
#define int long long
int n,x;
struct node{
    int val;
    node* lchild=NULL,*rchild=NULL;
};

```

```

void ist(node* root,int v){
    node* tmp=root,*add=new node;
    add->val=v;
    while (tmp!=NULL){
        if (v==tmp->val) return;
        else if (v>tmp->val){
            if (tmp->rchild==NULL) tmp->rchild=add;
            else tmp=tmp->rchild;
        }else{
            if (tmp->lchild==NULL) tmp->lchild=add;
            else tmp=tmp->lchild;
        }
    }
}

void preorder(node* nd){
    if (nd==NULL) {cout<<0<<" ";return;}
    cout<<nd->val<<" ",preorder(nd->lchild),preorder(nd->rchild);
}

signed main(){
    cin>>n>>x;
    node* root=new node;
    root->val=x;
    for (int i=2;i<=n;i++) cin>>x,ist(root,x);
    preorder(root);
    return 0;
}

```

G 乘 x 乘

这道题是 2023 级计算机学院《数据结构与算法》期末考试的一道编程题，整个计算机学院的学生中仅有 1 人 AC，得到 9 分（满分 15 分）以上的学生不超过 10%。

容易想到的方法是，先用题目提示中的方法封装一个结构体，然后用堆去模拟。但由于 K 的规模可以达到 10^9 ，这样做显然是不可行的。

因此我们考虑更优的做法。由于最小值一直在乘 q ，也就是呈指数型增长。那么直观上，很容易感受到，在操作次数足够多之后，最初数组中 a 的值几乎对排序的结果没有影响了。如果某次操作的时候我们发现，堆顶的元素乘以 q 之后，超过了原来堆中最大的元素，

那么接下来的每一次操作，优先队列的队头出队，乘以 q 之后再入队，一定会在队尾. 这样就构成了一个循环，直到操作次数耗尽为止.

考虑最坏的情况, n 的规模达到 2×10^4 , 序列中只有一个 10^9 , 其他数全部为 1, $q = 2$. 这种情况下，达到上面所说的那种情况，消耗的时间是最多的. 由于 $2^{30} \approx 10^9$, 所以每个数的操作次数约为 30 次，总操作次数可以控制在 3×10^5 之内，可以在 1s 的时间内运行通过.

达到上述情况后，问题就变得很简单了，先计算 $left = K \% n$ ，那么前 $left$ 个数被操作的次数为 $K/n + 1$ ，剩余的数被操作的次数为 K/n ，此时花费 $O(n)$ 的时间计数即可.

```
#include<bits/stdc++.h>
using namespace std;
#define int long long
int n,d,K,cnt[200005];
int ksm(int a,int b){
    int ans=1;
    while(b>0){
        if(b&1) ans*=a;
        a=a*a,b>>=1;
    }
    return ans;
}
struct node{
    int a=0,k=0,idx=0;
    bool operator < (const node& nd) const{
        if (k>=nd.k){
            if (a*ksm(d,k-nd.k)==nd.a) return idx>nd.idx;
            return a*ksm(d,k-nd.k)>nd.a;
        }else{
            if (a==nd.a*ksm(d,nd.k-k)) return idx>nd.idx;
            return a>nd.a*ksm(d,nd.k-k);
        }
    }
};
priority_queue<node,vector<node>> q;
node mx;
signed main(){
    ios::sync_with_stdio(false);
```

```

cin.tie(nullptr);
cout.tie(nullptr);
cin>>n>>d>>K;
int x;node nd;
for (int i=1;i<=n;i++){
    cin>>x,nd.a=x,nd.k=0,nd.idx=i;
    q.push(nd),mx=min(mx,nd);
}
while (K--){
    nd=q.top(),q.pop();
    nd.k++,cnt[nd.idx]++,q.push(nd);
    if (nd<mx) break;
}
int lft=K%n;
while (lft--) nd=q.top(),q.pop(),cnt[nd.idx]+=K/n+1;
while (!q.empty()) nd=q.top(),q.pop(),cnt[nd.idx]+=K/n;
for (int i=1;i<n;i++) cout<<cnt[i]<<" ";
cout<<cnt[n]<<endl;
return 0;
}

```

H 修理牧场

只要看出这就是一个哈夫曼树板子，就很好解决了。

```

#include <iostream>
#include <queue>
#include <vector>
using namespace std;
#define int long long
int n, x, wpl = 0, tmp1, tmp2;
struct cmp {
    bool operator()(const int &x1, const int &x2) {
        return x1 > x2;
    }
};

```

```

priority_queue<int, vector<int>, cmp> q;
signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cout.tie(nullptr);
    cin >> n;
    for (int i = 1; i <= n; i++)
        cin >> x, q.push(x);
    while (q.size() > 1) {
        tmp1 = q.top(), q.pop();
        tmp2 = q.top(), q.pop();
        wpl += (tmp1 + tmp2);
        q.push(tmp1 + tmp2);
    }
    cout << wpl << endl;
    return 0;
}

```

I 运动会与塑料基友

这题最开始的数据范围是 $n \leq 10^9, m \leq 10^6$. 本来只有第三种方法是可以通过全部测试点的, 但是为了减小这题的难度和通过人数, 遂减小了数据, 把 $O(n)$ 和 $O(n \log n)$ 的做法都放过了.

第一种方法: 维护一个大小不超过 k 的最小堆. 双指针滑动窗口去扫, 右指针遇到某右端点时, 将其对应的所有左端点全部入堆. 如果堆的大小超过了 k , 那就右移左指针, 不断移除堆顶元素. 需要注意的是, 如果以某个位置为左端点的基友对有多个, 那么在移除的时候, 它们应当一起被移除 (尽管可能移除部分之后, 堆的大小已经小于等于 k 了). 时间复杂度 $O(n \log n)$.

```

#include<iostream>
#include<queue>
using namespace std;
#define int long long
struct cmp{
    bool operator()(const int& x1,const int& x2){
        return x1>x2;
    }
}

```

```

    }
};
int n,m,k,x,y,ans=0;
vector<int> r1[1000005];
priority_queue<int,vector<int>,cmp> q;
signed main(){
    cin>>n>>m>>k;
    for (int i=1;i<=m;i++){
        cin>>x>>y;
        if (x>y) swap(x,y);
        r1[y].push_back(x);
    }
    int ii=1;
    for (int i=1;i<=n;i++){
        while (ii<=n&&q.size()<=k){
            for (size_t j=0;j<r1[ii].size();j++){
                if (r1[ii][j]>=i) q.push(r1[ii][j]);
            }
            ii++;
        }
        if (q.size()>k) ans+=(ii-i-1);
        else ans+=(ii-i);
        while (!q.empty()&&q.top()==i) q.pop();
    }
    cout<<ans<<endl;
    return 0;
}

```

第二种方法：对每个位置，用两个 vector，分别存以该位置为左端点的所有右端点，和以该位置为右端点的所有左端点。然后滑动窗口从左往右扫，右指针右移直到区间内的塑料基友对数大于 k ，此时以左指针所在位置为左端点的合法区间数量是可以计算的，累加答案，左指针右移，以此类推。时间复杂度 $O(n)$ 。

```

#include<iostream>
using namespace std;
#define int long long
int n,m,k,x,y,ans=0,tmp=0;

```

```

vector<int> lr[1000005],rl[1000005];
signed main(){
    cin>>n>>m>>k;
    for (int i=1;i<=m;i++){
        cin>>x>>y;
        if (x>y) swap(x,y);
        lr[x].push_back(y),rl[y].push_back(x);
    }
    int ii=1;
    for (int i=1;i<=n;i++){
        while (ii<=n&&tmp<=k){
            for (size_t j=0;j<rl[ii].size();j++){
                if (rl[ii][j]>=i) tmp++;
            }
            ii++;
        }
        if (tmp>k) ans+=(ii-i-1);
        else ans+=(ii-i);
        for (size_t j=0;j<lr[i].size();j++){
            if (lr[i][j]<=ii) tmp--;
        }
    }
    cout<<ans<<endl;
    return 0;
}

```

第三种方法：最优解，窗口不是一点一点滑动而是跳动。时间复杂度 $O(m \log m)$. 可以通过 $n \leq 10^9, m \leq 10^6$ 的数据。出题的时候本身想这样出的，但是觉得这个做法对大家有些过于困难了，遂减小了数据量，把前两种做法都放过了。我的写法不太优美，这里贴个糕手的代码，大家可以借鉴学习。

```

#include<algorithm>
#include<iostream>
#include<iomanip>
#include<cstring>
#include<cstdio>
#include<vector>

```

```

#include<cmath>
#include<queue>
#include<map>
#define maxn 100005
using namespace std;
typedef long long ll;
int read() {
    int x = 0, f = 1, ch = getchar();
    while(!isdigit(ch)) {if(ch == '-') f = -1; ch = getchar();}
    while(isdigit(ch)) x = (x << 1) + (x << 3) + ch - '0', ch = getchar();
    return x * f;
}
int n, m, k;
struct node {
    int l, r, id;
}a[maxn];
bool cmp(node a, node b) {return a.r == b.r? a.l < b.l : a.r < b.r;}
priority_queue<pair<int, int> > q;
signed main() {
    n = read(), m = read(), k = read();
    for(int i = 1; i <= m; i++) {
        a[i].l = read(), a[i].r = read();
        a[i].id = i;
    }
    sort(a + 1, a + 1 + m, cmp);
    a[m + 1] = {n + 1, n + 1, m + 1};
    ll ans = 0;
    int pre = 0;
    //1~a[1].r-1的部分特殊处理。这个肯定全都是合法的
    for(int i = 1; i <= m; i++) {//以a[i].r ~ a[i+1].r-1为右端点的合法区间数
        q.push(make_pair(-a[i].l, -a[i].r));
        while(a[i].r == a[i + 1].r) i++, q.push(make_pair(-a[i].l, -a[i].r));
        while(q.size() > k || pre == -q.top().first) {
            pre = -q.top().first, q.pop();
            if(!q.size()) break;
        }
    }
}

```

```

        int L = a[i].r - pre, R = a[i + 1].r - pre - 1;
        ans += 1ll * (L + R) * (a[i + 1].r - a[i].r) / 2;
    }
    cout << ans + 1ll * a[1].r * (a[1].r - 1) / 2 << endl;
    return 0;
}

```

J 先輩からのムチ、愛をこめて

这是去年弘深班期末考试的第三题。

首先用堆进行暴力维护可以获得 15 分的暴力分。但这样的时间复杂度是 $O(\sum a_i)$ 的，我们考虑优化。

- 如果 a_i 的最大值超过了剩余所有值的和，那么最优策略显然是每次都选中这个值和剩余的任意一个值。最后再将这个值归零。那么答案就是这个最大值。
- 否则，我们按照上述暴力的策略进行操作，一定不会有剩余（或者最后剩一个 1），答案为 $\frac{\text{sum}+1}{2}$ 。

```

#include<iostream>
using namespace std;
#define int long long
int n,lst[1000005],mx=0,sum=0;
signed main(){
    cin>>n;
    for (int i=1;i<=n;i++) cin>>lst[i],sum+=lst[i],mx=max(mx,lst[i]);
    if (sum-mx<=mx) cout<<mx<<endl;
    else cout<<(sum+1)/2<<endl;
    return 0;
}

```